La gestion des branches

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Avantages de la gestion de branches **4**

A. Avantages de la gestion de branches 4

B. Exercice : Quiz 6

III - La gestion des branches Git en pratique **8**

A. La gestion des branches Git en pratique 8

B. Exercice : Quiz 10

IV - Essentiel **13**

V - Auto-évaluation **14**

A. Exercice 14

B. Test 14

Solutions des exercices **16**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Environnement de travail : PC

Prérequis : notions de Git

Contexte

Vous avez découvert Git, ce logiciel de versionnage performant, devenu indispensable pour la plupart des développeurs. Vous savez *commiter*, c'est-à-dire faire un Git commit, une action qui sauvegarde des modifications qui se trouvent dans la zone index et qui seront stockées au niveau du repository. À présent, il faut approfondir ce logiciel puissant et découvrir les branches.

La gestion des branches Git est un aspect essentiel et puissant du système de contrôle de versions Git qui facilite le développement collaboratif, la gestion de plusieurs fonctionnalités et la maintenance d'un code stable et fonctionnel.

Git est un système de contrôle de versions distribué. Il est utilisé pour suivre l'évolution du code source d'un projet. Il permet à plusieurs développeurs de travailler simultanément sur un projet, tout en maintenant un historique complet des modifications apportées au code.

Dans ce cours, nous allons expliquer les concepts clés de la gestion des branches Git, l'importance des branches et la manière dont elles aident les équipes de développement à travailler ensemble de manière plus efficace et organisée.

Tous droits réservés à STUDI - Reproduction interdite

II Avantages de la gestion de branches

A. Avantages de la gestion de branches

Remarque

Git est un système de contrôle de version décentralisé et puissant. L'un de ses avantages est la gestion de branches, qui permet aux développeurs de travailler sur des fonctionnalités ou des correctifs de bugs sans affecter la branche principale ou la version de production du projet. Cette fonctionnalité est très importante pour plusieurs raisons.

Les nombreux avantages

L'une des principales raisons pour lesquelles la gestion des branches est importante est **l'isolation des modifications**. En créant une nouvelle branche pour travailler sur une fonctionnalité ou un correctif de bug, les développeurs peuvent travailler sur leur code sans affecter la branche principale ou la version de production du projet. Cela permet de développer et de tester des fonctionnalités ou des modifications de manière isolée, ce qui peut améliorer la qualité globale du code.

Lorsque plusieurs développeurs travaillent sur un même projet, il peut y avoir des conflits entre les modifications qu'ils apportent. En utilisant des branches distinctes pour travailler sur des fonctionnalités spécifiques ou des corrections de bug, chaque développeur peut travailler de manière isolée, sans affecter le travail des autres. Les branches permettent de séparer les modifications en cours de développement de la branche principale du projet, ce qui rend la gestion des modifications plus facile et plus sûre.

Grâce aux branches Git, il est également possible de travailler sur des **versions stables et instables** de l'application. Les développeurs peuvent créer une branche stable qui contient le code de la version actuelle de l'application, tandis que les modifications sont effectuées sur une branche instable. Cela permet de travailler sur de nouvelles fonctionnalités ou améliorations sans risquer de perturber la version actuelle de l'application. Une fois les modifications effectuées et validées, elles peuvent être fusionnées avec la branche stable, créant ainsi une nouvelle version stable de l'application.

En utilisant des branches, vous pouvez également **suivre les différentes versions de votre projet** au fil du temps. Cela facilite le suivi des modifications et donne une vision globale de l'évolution d'un projet ou d'une application web. Cela peut aider à repartir en arrière pour démarrer avec de nouvelles branches sur d'autres besoins.

L'organisation des branches grâce à Git **facilite les tests et la validation du code** en permettant de travailler sur une branche créée directement sur une *feature*, ou fonctionnalité, elle-même sur une branche. Une fois les tests et les validations effectués avec succès, les modifications peuvent être intégrées au code principal en toute sécurité.

La supervision des branches *via* Git permet de **revenir en arrière** sur une modification ou une fonctionnalité si nécessaire. Les branches permettent de conserver une trace de toutes les modifications effectuées dans le code, ce qui permet de revenir en arrière sur une branche précédente si une modification ne fonctionne pas ou si elle cause des problèmes. Cela permet de réduire les risques liés aux changements dans le code et de faciliter la maintenance du code à long terme. De plus, la gestion des branches permet aux développeurs de travailler en toute confiance en sachant qu'ils peuvent toujours revenir en arrière sur une modification si nécessaire.

Tous droits réservés à STUDI - Reproduction interdite

Une **collaboration efficace** entre les membres d'une équipe est un autre avantage qu'apporte le système de branches Git, en offrant un moyen clair de suivre et de gérer les modifications apportées au code source. En effet, chaque collaborateur peut travailler sur une branche différente d'un même projet, avec l'objectif de réintégrer la branche maîtresse. La collaboration simplifie la gestion des tâches en créant des branches pour des fonctionnalités spécifiques, des correctifs ou des améliorations, facilitant ainsi la révision du code et les retours d'information entre les membres de l'équipe travaillant sur le projet.

Les branches Git permettent aux développeurs **d'expérimenter de nouvelles fonctionnalités** ou des améliorations sans risquer de perturber le code principal. Les développeurs peuvent créer une branche dédiée pour chaque nouvelle fonctionnalité, tester et valider la nouvelle fonctionnalité indépendamment du reste du code, avant de la fusionner avec le code principal.

Un autre avantage concerne les *releases*. L'administration des branches avec Git permet de gérer les releases, c'est-à-dire les **versions finales du logiciel** qui sont mises à disposition des utilisateurs. Les développeurs peuvent créer une branche dédiée pour chaque release, ce qui permet de conserver une trace de l'historique des modifications apportées à la release et de corriger rapidement les éventuels bugs ou problèmes rencontrés par les utilisateurs.

De plus, le contrôle des branches Git permet d'implémenter des **pratiques de déploiement continu**, qui consistent à automatiser le processus de déploiement des nouvelles fonctionnalités ou des corrections de bugs. Les développeurs peuvent travailler sur des branches dédiées à chaque fonctionnalité ou tâche, et une fois le code validé, il est automatiquement déployé en production. Cela permet de réduire les temps de déploiement, d'optimiser le processus de livraison et de minimiser les risques d'erreur.

Enfin, la manipulation des branches dans Git offre une **grande flexibilité** aux développeurs, qui peuvent créer autant de branches qu'ils le souhaitent pour travailler sur différentes parties du code ou essayer de nouvelles possibilités. Puisqu'il est possible d'ajouter ou d'élaguer des branches facilement, un projet devient rapidement souple et polyvalent.

Sauvegarde du code

Conseil

Sauvegarder le code à différentes étapes du développement est une étape essentielle pour un programmeur. En effet, cela permet de garder un historique complet des modifications apportées. Cela facilite la compréhension de l'évolution du projet. Les concepteurs de logiciels peuvent créer une branche pour chaque étape importante du développement et avoir une excellente traçabilité.

Git log

La commande Git `'git log'` permet d'afficher l'historique des commits d'un dépôt. Cette commande vous offre la possibilité de visualiser les détails des commits, tels que l'auteur, la date et le message de commit, ainsi que l'identifiant unique (SHA) de chaque commit. L'historique des commits est affiché par ordre chronologique inverse, avec le commit le plus récent en premier.

Voici quelques options courantes pour personnaliser l'affichage de git log :

- `--oneline` : affiche chaque commit sur une seule ligne, avec un format condensé incluant l'identifiant SHA et le message de commit.
- `--graph` : permet d'afficher l'historique des commits sous forme de graphique en texte, montrant la structure des branches et les fusions.
- `--decorate` : ajoute des informations supplémentaires, telles que les noms de branches et les tags, aux commits.
- `--all` : donne l'historique des commits pour toutes les branches, et pas seulement la branche actuelle.

Tous droits réservés à STUDI - Reproduction interdite

- `--author=<auteur>` : affiche seulement les commits créés par un auteur spécifique.
- `--since=<date>` ou `--after=<date>` : affiche les commits effectués après une date spécifique.
- `--until=<date>` ou `--before=<date>` : renseigne sur les commits effectués avant une date spécifique.

Méthode

Pour utiliser ces options, il suffit de les ajouter après la commande `git log`. Par exemple, pour afficher un graphique en texte de l'historique des commits avec une ligne par commit, vous pouvez utiliser :

```
$ git log --graph --oneline --decorate --all
```

`git log` est un outil puissant pour explorer l'historique des commits, identifier les changements apportés au code et comprendre l'évolution d'un projet.

B. Exercice : Quiz

[solution n°1 p. 16]

Question 1

Qu'est-ce que la gestion des branches de Git ?

- La création d'un nouveau repository
- La gestion des différentes versions du code
- La gestion des conflits entre les développeurs

Question 2

Quel est l'un des avantages de la gestion des branches de Git ?

- L'élimination des conflits de fusion
- La réduction du nombre de développeurs sur un projet
- L'isolation des modifications

Question 3

Quel est l'avantage de travailler sur une branche isolée ?

- Cela permet de simplifier la gestion des tâches
- Cela permet de tester et de développer des fonctionnalités ou des modifications
- Cela permet de réduire les risques d'erreur

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Pourquoi est-il important de travailler avec des branches sur des versions stables et instables de l'application ?

- Pour pouvoir créer une branche dédiée pour chaque release
- Pour travailler sur de nouvelles fonctionnalités ou améliorations sans risquer de perturber la version actuelle de l'application
- Pour permettre aux développeurs d'expérimenter de nouvelles fonctionnalités ou des améliorations

Question 5

Comment la gestion des branches de Git permet-elle d'expérimenter de nouvelles fonctionnalités ?

- En faisant des commits régulier
- En switchant sur les branches
- En créant une branche dédiée pour chaque nouvelle fonctionnalité

Tous droits réservés à STUDI - Reproduction interdite

III La gestion des branches Git en pratique

A. La gestion des branches Git en pratique

Git est souvent comparé à un arbre, car il suit une structure de type « *arborescence* » pour gérer les versions de code. Lorsque vous créez un nouveau dépôt Git, cela crée une « *racine* » qui représente l'état initial de votre projet. Chaque modification que vous apportez au code est enregistrée sous forme de commit, qui est un instantané de l'état de votre code à un moment donné. Ces commits sont organisés en une séquence linéaire, appelée la « *branche principale* » ou la « *branche maître* » (*master branch*).

Notez que chaque branche doit commencer à partir d'un commit existant et peut avoir sa propre séquence de commits.

Naviguer dans le terminal

Pour naviguer dans le terminal, vous pouvez utiliser les commandes suivantes :

- `cd` : cette commande vous permet de changer de répertoire. Par exemple, si vous voulez vous déplacer dans le dossier « *Documents* », vous pouvez taper `cd Documents`. Si vous voulez remonter d'un niveau, vous pouvez utiliser `cd ..`.
- `ls` : cette instruction autorise à lister le contenu du répertoire dans lequel vous vous trouvez. Par exemple, si vous êtes dans le dossier « Documents », vous pouvez taper `ls` pour voir tous les fichiers et sous-dossiers présents dans ce dossier.
- `mkdir` : grâce à cette commande, vous pouvez créer un nouveau dossier. Par exemple, si vous voulez créer un nouveau dossier appelé « *Nouveau dossier* », vous pouvez taper `mkdir "Nouveau dossier"`.
- `touch` : cette commande offre la possibilité de créer un nouveau fichier vide. Par exemple, si vous voulez créer un nouveau fichier appelé « *nouveau_fichier.txt* », vous pouvez taper `touch nouveau_fichier.txt`.
- `rm` : en utilisant cette commande, vous êtes en mesure de supprimer un fichier ou un dossier. Par exemple, si vous voulez supprimer un fichier appelé « *mon_fichier.txt* », vous pouvez taper `rm mon_fichier.txt`. Si vous voulez supprimer un dossier appelé « *Mon dossier* », vous pouvez utiliser `rm -r "Mon dossier"`.
- pwd : cette commande permet de voir le chemin complet du répertoire dans lequel vous vous trouvez actuellement.
- `history` : cette commande donne accès à l'historique des commandes que vous avez utilisées dans le terminal.
- `clear` : l'usage de cette commande vous habilite à vider l'écran du terminal.

Ces commandes de base devraient vous permettre de naviguer et de travailler efficacement dans le terminal.

Créer une branche

Méthode

Pour créer une nouvelle branche avec Git, vous pouvez suivre les étapes suivantes.

Ouvrez votre terminal ou invite de commande et accédez au répertoire de votre projet Git.

Assurez-vous que vous êtes sur la branche principale (généralement « *master* » ou « *main* ») en utilisant la commande :

```
1 $ git status
```


Tous droits réservés à STUDI - Reproduction interdite

La commande `git status` permet de vérifier l'état actuel de votre répertoire de travail par rapport à votre référentiel Git local. Elle affiche les modifications apportées à vos fichiers, les fichiers qui ont été modifiés, mais qui ne sont pas encore validés, les fichiers qui ont été ajoutés ou supprimés, mais qui n'ont pas encore été validés, et les fichiers qui ont été validés, mais qui n'ont pas encore été poussés vers le référentiel distant. En d'autres termes, cette commande permet de savoir si votre répertoire de travail est propre ou s'il y a des modifications ou des ajouts qui doivent être validés ou poussés.

Créez une nouvelle branche en utilisant la commande `git branch` suivie du nom de votre nouvelle branche. Par exemple, si vous voulez créer une branche appelée « *ma-branche* », vous pouvez taper `git branch ma-branche`.

```
$ git branch “ma-branche”
```

Git vous permet également de créer des branches sur des branches pour travailler sur des fonctionnalités ou des tâches spécifiques de manière isolée.

L'ensemble de ces branches (et des commits associés) peut être représenté sous forme d'arborescence, où chaque commit est un nœud et chaque branche est une branche de l'arbre. Cette structure en arborescence vous permet de visualiser facilement l'historique des versions de votre code, d'isoler des modifications pour les tester ou les vérifier, de collaborer avec d'autres développeurs sur des fonctionnalités spécifiques et de revenir en arrière à tout moment.

Git checkout

La commande `git checkout` est une commande clé de Git qui permet de basculer entre différentes branches ou de récupérer des fichiers individuels depuis une branche spécifique. Cette commande peut être utilisée pour effectuer les opérations suivantes :

Utilisation de Git checkout

Méthode

Git checkout permet de passer d'une branche à une autre en utilisant la commande suivante :

```
$ git checkout nom-de-la-branche
```

Par exemple, si vous voulez passer de la branche « *feature* » à la branche « *main* », vous pouvez exécuter la commande `git checkout main`.

Créer une nouvelle branche : `git checkout` peut également être utilisé pour créer une nouvelle branche et y basculer en une seule commande. Pour cela, utilisez la commande suivante :

```
$ git checkout -b nom-de-la-nouvelle-branche
```

Par exemple, si vous voulez créer une nouvelle branche appelée « *ma-nouvelle-branche* » à partir de la branche actuelle, vous pouvez exécuter la commande ci-dessus.

Vous pouvez également créer une branche enfant d’une branche parent à partir de n’importe quelle branche existante, même si vous êtes sur la branche master.

```
$ git checkout -b nom-branche-enfant branche-existante
```

Par exemple, vous étiez sur la branche « *main* », mais avec la commande ci-dessus vous vous retrouverez sur la branche « *nom-branche-enfant* » qui vient d’être créée.

Pour connaître la liste des branches existantes dans votre projet et savoir sur quelle branche vous êtes, tapez cette ligne de commande :

```
$ git branch --list
```

Vous aurez ainsi dans le terminal la liste de toutes vos branches en blanc et en vert avec un astérisque devant la branche sur laquelle vous êtes.

Tous droits réservés à STUDI - Reproduction interdite

Notez que la commande vous permet de basculer sur la dernière branche ou vous étiez :

```
$ git checkout -
```

Par exemple, avec la commande ci-dessus, si à partir de la branche « *main* » vous aviez créé une branche enfant d'une branche existe avec `git checkout`, vous reviendrez à la branche « *main* ».

Git switch

La commande `git switch` a été introduite dans la version 2.23 de Git, c'est une alternative à la commande `git checkout`. Elle permet de basculer facilement entre les branches et de créer de nouvelles branches en une seule commande. Elle est considérée par certains comme plus intuitive.

Méthode

La commande `git switch` offre les mêmes possibilités que `git checkout` et s'exécute de la même façon. Il suffit de remplacer « *checkout* » par « *switch* », à l'exception de `git checkout -b nom-de-la-branche` qui a pour équivalent `git switch -c nom-de-la-branche`.

```
$ git switch  -
$ git switch -c nouvelle-branche
```

La première ligne de commande permet de revenir sur la branche d'avant et la seconde de créer « *nouvelle-branche* » et de travailler dessus.

Supprimer une branche

Méthode

Supposons que vous avez créé une nouvelle branche appelée « *nouvelle_fonctionnalité* » pour travailler sur une fonctionnalité de votre projet et que vous ayez terminé votre travail sur cette branche. Vous pouvez alors supprimer cette branche en utilisant la commande `git branch -d` comme suit :

```
$ git branch -d nouvelle_fonctionnalité
```

Si la branche « *nouvelle_fonctionnalité* » a été fusionnée dans la branche actuelle, elle sera supprimée avec succès et vous verrez un message confirmant la suppression de la branche. Si la branche n'a pas été fusionnée, Git affichera un avertissement indiquant que la branche n'a pas été fusionnée et qu'elle ne sera pas supprimée avec `git branch -d`. Si vous êtes sûr de vouloir supprimer la branche même si elle n'a pas été fusionnée, vous pouvez utiliser la commande `git branch -D` :

```
$ git branch -D nouvelle_fonctionnalité
```

Cela supprime la branche « *nouvelle_fonctionnalité* » même si elle n'a pas été fusionnée, et vous verrez un message confirmant la suppression de la branche.

B. Exercice : Quiz

[solution n°2 p. 17]

Question 1

Comment créer une nouvelle branche dans Git ?

git create branch nom_de_la_branche

Tous droits réservés à STUDI - Reproduction interdite

git branch nom_de_la_branche

git new branch nom_de_la_branche

Question 2

Quelle est la différence entre `git branch -d` et `git branch -D` ?

git branch -d supprime une branche uniquement si elle a été fusionnée avec succès dans une autre branche, tandis que git branch -D supprime une branche locale de manière forcée

git branch -d supprime une branche fusionnée, tandis que git branch -D supprime une branche sans la fusionner

Il n'y a pas de différence entre git branch -d et git branch -D

Question 3

Quelle commande ne vous permet pas de savoir sur quelle branche vous êtes actuellement dans Git ?

git check

git branch --list

git status

Question 4

Comment pouvez-vous créer une nouvelle branche et basculer dessus en une seule commande dans Git ?

A git branch -b nom-de-la-nouvelle-branche

B git switch -b nom-de-la-nouvelle-branche

C git checkout -b nom-de-la-nouvelle-branche

Question 5

Comment pouvez-vous créer une branche enfant à partir d'une branche parent dans Git ?

git branch -b nom-branche-enfant branche-parent

git checkout -b nom-branche-enfant branche-parent

Tous droits réservés à STUDI - Reproduction interdite

```
git switch -b nom-branche-enfant branche-parent
```


Tous droits réservés à STUDI - Reproduction interdite

IV Essentiel

Git est un système de contrôle de version populaire qui suit une structure de type « *arborescence* » pour gérer les versions de code. Chaque modification apportée au code est enregistrée sous forme de commit, qui est un instantané de l'état de votre code à un moment donné. Git permet également de créer des branches pour travailler sur des fonctionnalités ou des tâches spécifiques de manière isolée. Chaque branche commence à partir d'un commit existant et peut avoir sa propre séquence de commits.

La commande `git checkout` est une commande clé de Git qui permet de basculer entre différentes branches ou de récupérer des fichiers individuels depuis une branche spécifique. `Git switch` est une alternative à la commande `git checkout` introduite dans la version 2.23 de Git. Elle permet de basculer facilement entre les branches et de créer de nouvelles branches en une seule commande.

Enfin, nous avons vu comment supprimer une branche avec les commandes `git branch -d` et `git branch -D`, qui permet de supprimer une branche même si elle n'a pas été fusionnée.

La structure en arborescence ou en forme d'arbre de Git permet de visualiser facilement l'historique des versions de votre code, d'isoler des modifications pour les tester ou les vérifier, de collaborer avec d'autres développeurs sur des fonctionnalités spécifiques et de revenir en arrière à tout moment.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous travaillez sur un projet en équipe avec plusieurs développeurs, et vous devez ajouter une nouvelle fonctionnalité à votre application. Vous voulez créer une nouvelle branche pour travailler sur cette fonctionnalité de manière isolée.

Question 1

[solution n°3 p. 19]

Effectuez les étapes suivantes :

- Ouvrez votre terminal ou invite de commande et accédez à MyProjet.
- Assurez-vous que vous êtes sur la branche principale.
- Créez une nouvelle branche « *ma-super-fonctionnalité* ».
- Basculez sur la nouvelle branche.
- Enregistrez les modifications.

Question 2

[solution n°4 p. 19]

Vous avez créé la branche « *ma-super-fonctionnalité* », cependant, vous devez maintenant créer une nouvelle branche nommée « *sous-fonctionnalité-1* » pour effectuer des tests supplémentaires sur une partie spécifique de « *ma-super-fonctionnalité* », tout en gardant le reste du code intact sur la branche « *ma-super-fonctionnalité* ». Comment allez-vous vous y prendre ?

B. Test

Exercice 1 : Quiz

[solution n°5 p. 20]

Question 1

Quelle commande est destinée à créer une branche ?

- git branch ma-branche
- git checkout ma-branche
- git switch ma-branche

Question 2

Pourquoi la gestion des branches de Git est-elle importante ?

- Pour la collaboration entre les membres de l'équipe
- Pour avoir un arbre
- Pour éviter les conflits entre les développeurs

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Avec les branches Git, il est possible de revenir en arrière sur une modification ou une fonctionnalité.

- Vrai
- Faux

Question 4

Pourquoi est-ce important de pouvoir revenir en arrière sur une modification dans le code ?

- Pour créer de nouvelles fonctionnalités
- Pour faciliter la maintenance du code à long terme
- Pour s'amuser

Question 5

`git switch` est une commande plus récente que `git checkout` pour changer de branche.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 6]

Question 1

Qu'est-ce que la gestion des branches de Git ?

- La création d'un nouveau repository
- ✓ La gestion des différentes versions du code
- La gestion des conflits entre les développeurs

La gestion des branches de Git permet de travailler sur différentes versions du code simultanément.

Question 2

Quel est l'un des avantages de la gestion des branches de Git ?

- L'élimination des conflits de fusion
- La réduction du nombre de développeurs sur un projet
- ✓ L'isolation des modifications

L'avantage de l'isolation des modifications est que la branche ne peut nuire au reste du code.

Question 3

Quel est l'avantage de travailler sur une branche isolée ?

- Cela permet de simplifier la gestion des tâches
- ✓ Cela permet de tester et de développer des fonctionnalités ou des modifications
- Cela permet de réduire les risques d'erreur

Tous droits réservés à STUDI - Reproduction interdite

En travaillant sur une branche isolée, il est possible de développer et de tester des fonctionnalités ou des modifications sans affecter la branche principale.

Question 4

Pourquoi est-il important de travailler avec des branches sur des versions stables et instables de l'application ?

Pour pouvoir créer une branche dédiée pour chaque release

✓ Pour travailler sur de nouvelles fonctionnalités ou améliorations sans risquer de perturber la version actuelle de l'application

Pour permettre aux développeurs d'expérimenter de nouvelles fonctionnalités ou des améliorations

Que la version de l'application soit stable ou instable, pour travailler sur de nouvelles fonctionnalités, il faut créer une nouvelle branche afin de ne pas risquer de perturber la version actuelle de l'application.

Question 5

Comment la gestion des branches de Git permet-elle d'expérimenter de nouvelles fonctionnalités ?

En faisant des commits régulier

En switchant sur les branches

✓ En créant une branche dédiée pour chaque nouvelle fonctionnalité

Git permet de créer une branche dédiée pour chaque nouvelle fonctionnalité.

Solution n°2

[exercice p. 10]

Question 1

Comment créer une nouvelle branche dans Git ?

git create branch nom_de_la_branche

✓ git branch nom_de_la_branche

Tous droits réservés à STUDI - Reproduction interdite

git new branch nom_de_la_branche

Pour créer une nouvelle branche dans Git, vous devez utiliser la commande `git branch ma_branche` par exemple.

Question 2

Quelle est la différence entre `git branch -d` et `git branch -D` ?

✓ git branch -d supprime une branche uniquement si elle a été fusionnée avec succès dans une autre branche, tandis que git branch -D supprime une branche locale de manière forcée

git branch -d supprime une branche fusionnée, tandis que git branch -D supprime une branche sans la fusionner

Il n'y a pas de différence entre git branch -d et git branch -D

La différence entre les deux commandes est que « *git branch –D* » supprime une branche même si elle n'a pas été fusionnée.

Question 3

Quelle commande ne vous permet pas de savoir sur quelle branche vous êtes actuellement dans Git ?

git check

✓ git branch --list

git status

Cette commande n'existe pas. `git branch --list` vous liste toutes les branches du projet et vous affiche en vert la branche sur laquelle vous travaillez, tandis que `git status` vous permet de vérifier l'état actuel de votre répertoire et vous indique sur quelle branche vous êtes.

Question 4

Comment pouvez-vous créer une nouvelle branche et basculer dessus en une seule commande dans Git ?

A git branch -b nom-de-la-nouvelle-branche

B git switch -b nom-de-la-nouvelle-branche

Tous droits réservés à STUDI - Reproduction interdite

C git checkout -b nom-de-la-nouvelle-branche

git checkout -b nom-de-la-nouvelle-branche

Question 5

Comment pouvez-vous créer une branche enfant à partir d'une branche parent dans Git ?

✓ git branch -b nom-branche-enfant branche-parent

git checkout -b nom-branche-enfant branche-parent

git switch -b nom-branche-enfant branche-parent

Pour créer une branche avec `git checkout`, il faut ajouter l'option « *-b* » et avec `git switch l'option -c`.

[exercice p. 14]

Solution n°3

Ouvrez le terminal puis naviguez avec les commandes `ls`, `cd`, pour vous situer dans le répertoire MyProjet.

Tapez cette commande pour savoir sur quelle branche vous êtes :

```
$ git status
```

Puis allez sur la branche principale :

```
$ git checkout main
```

Créez votre nouvelle branche ainsi :

```
$ git branch ma-super-fonctionnalité
```

Puis vous basculez avec :

```
$ git checkout  ma-super-fonctionnalité
```

Et enfin, vous travaillez sur le projet et vous faites un `git commit`.

[exercice p. 14]

Solution n°4

Vous pouvez, comme dans la question 1, faire plusieurs manipulations, mais à partir de votre application, vous pouvez la créer (quelle que soit la branche dans laquelle vous êtes) en utilisant la commande suivante :

```
$ git checkout -b sous-fonctionnalité-1  ma-super-fonctionnalité
```

Tous droits réservés à STUDI - Reproduction interdite

Puis travaillez sur la branche « *sous-fonctionnalité-1* » en effectuant des commits réguliers. Lorsque vous aurez terminé tous vos tests et amélioré la fonctionnalité, vous fusionnez avec la branche « *ma-super-fonctionnalité* » (commande que nous n'avons volontairement pas décrit dans ce cours) et vous pourrez ensuite supprimer la branche « *sous-fonctionnalité-1* » avec cette commande :

```
$ git branch -d sous-fonctionnalité-1
```

Solution n°5

[exercice p. 14]

Question 1

Quelle commande est destinée à créer une branche ?

- ✓ git branch ma-branche
- git checkout ma-branche
- git switch ma-branche

`git checkout` et `git switch` permettent de basculer sur une branche déjà créée.

Question 2

Pourquoi la gestion des branches de Git est-elle importante ?

- ✓ Pour la collaboration entre les membres de l'équipe
- Pour avoir un arbre
- Pour éviter les conflits entre les développeurs

La gestion des branches de Git permet à chaque développeur de travailler sur sa propre branche, ce qui favorise la collaboration et la communication entre les membres de l'équipe.

Question 3

Avec les branches Git, il est possible de revenir en arrière sur une modification ou une fonctionnalité.

- ✓ Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Le système de branche permet de revenir en arrière avec une grande facilité.

Question 4

Pourquoi est-ce important de pouvoir revenir en arrière sur une modification dans le code ?

Pour créer de nouvelles fonctionnalités

✓ Pour faciliter la maintenance du code à long terme

Pour s'amuser

Revenir en arrière sur une modification dans le code permet de réduire les risques liés aux changements dans le code et de faciliter la maintenance du code à long terme.

Question 5

`git switch` est une commande plus récente que `git checkout` pour changer de branche.

✓ Vrai

Faux

`git switch` est plus récente et moderne, mais les 2 commandes sont utilisables à volonté.

Tous droits réservés à STUDI - Reproduction interdite